

$$2^m = n \quad (13.4)$$

where n is the number of inputs (outputs) to the MUX (DEMUX) and m is the number of control lines. A 4-to-1 MUX/DEMUX is shown in Fig. 13.8. Note that the MUX is bi-directional; that is, it can be used as a MUX or a DEMUX. The logic equation describing the operation of the MUX is given by

$$Z = A(S1 \cdot S2) + B(S1 \cdot \overline{S2}) + C(\overline{S1} \cdot S2) + D(\overline{S1} \cdot \overline{S2}) \quad (13.5)$$

Figure 13.9 shows the transistor-level implementation of the circuit in Fig. 13.8. Notice how the PMOS are grouped together (and laid out in the same n-well) while the NMOS are grouped together.

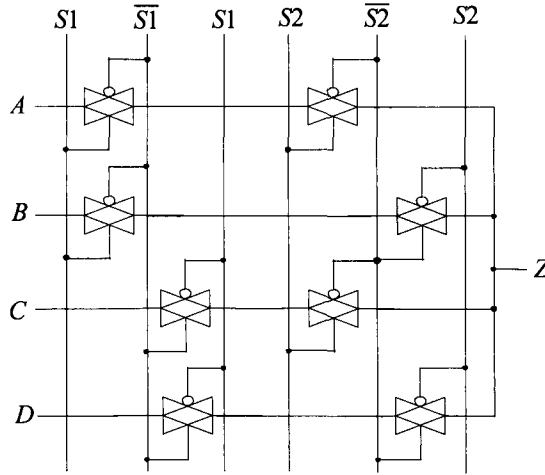


Figure 13.8 Circuit implementation of a 4-to-1 MUX.

Figure 13.10a shows an NMOS PG implementation of the 4-to-1 MUX. The pass transistor implementation is simpler, using fewer transistors, at the price of a threshold voltage drop from input to output when the input is a high (V_{DD}). A simplified version of the circuit of Fig. 13.10a is shown in Fig. 13.10b. Here the MOSFETs connected to $S2$ and $\overline{S2}$ are combined to reduce the total number of MOSFETs used. The reduction of the total number of MOSFETs used can be extended to an n -input (output) MUX (DEMUX) (see Fig. 16.43). Again, it should be remembered that a DEMUX can be formed using the circuits of Figs. 13.8 or 13.10 by switching the inputs with the outputs.

Static Gates

The TG can be used to form static logic gates. Consider the OR gate shown in Fig. 13.11. To understand the operation of the gate, consider the case when both A and B are low. Under these circumstances the pass transistor, M1 is off, and the TG is on. Since the input B is low, a low is passed to the output. If A is high, M1 is on and A is passed to the output. If B is high and A is low, B is passed to the output through the TG. If both A and B are high, the TG is off and M1 is on passing A , a high, to the output.

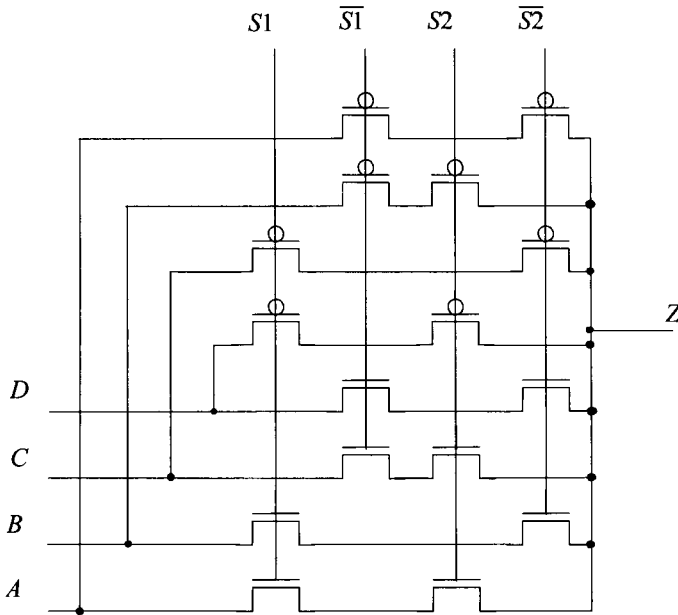


Figure 13.9 Transistor implementation of Fig. 13.8.

Figure 13.12 shows an XOR and an XNOR gate made using TGs. Consider the XOR gate with both A and B low. Under these circumstances, the top TG is on and its output is connected to A , a low. If both inputs are high, the bottom TG connects the output to $\bar{A}$, again a low. If A is high and B is low, the top TG is on and the output is connected to A , a high. Similarly, if A is low and B is high, the bottom TG is on and connects the output to $\bar{A}$, a high.

13.3 Latches and Flip-Flops

Basic Latches

Consider the set-reset latch (SR latch) shown in Fig. 13.13 made using NAND gates. The logic symbol and truth table are also shown in this figure. Consider the case when S is high and R is low. Forcing R low causes Q to go high. Since S is high and Q is high, the $\bar{Q}$ output is low. Now consider the case when both S and R are low. Under these circumstances, the latch's outputs are both high. This latch can easily be designed and laid out with the techniques of Ch. 12 (see also Fig. 15.6).

An alternative implementation of the SR latch is shown in Fig. 13.14 using NOR gates. Consider the case when S is high and R is low. For the NOR gate, a high input forces the output of the gate low. Therefore, the $\bar{Q}$ output is low whenever the S input is high. Similarly, whenever the R input is high, the Q output must be low. The case of both inputs being high causes both Q and $\bar{Q}$ to go low, or in other words the outputs of the latch are no longer complements.

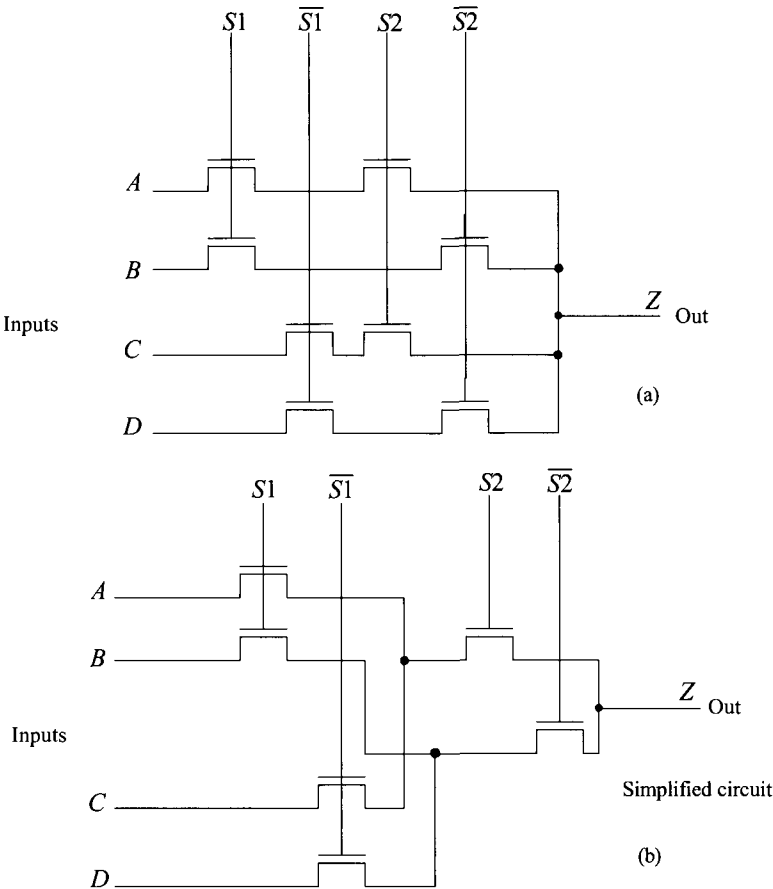


Figure 13.10 MUX/DEMUX using pass transistors.

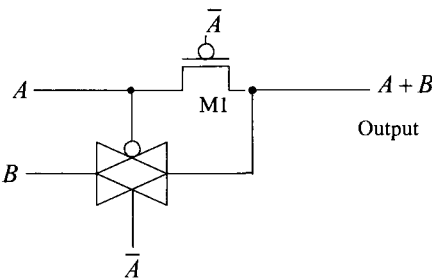


Figure 13.11 TG-based OR gate.

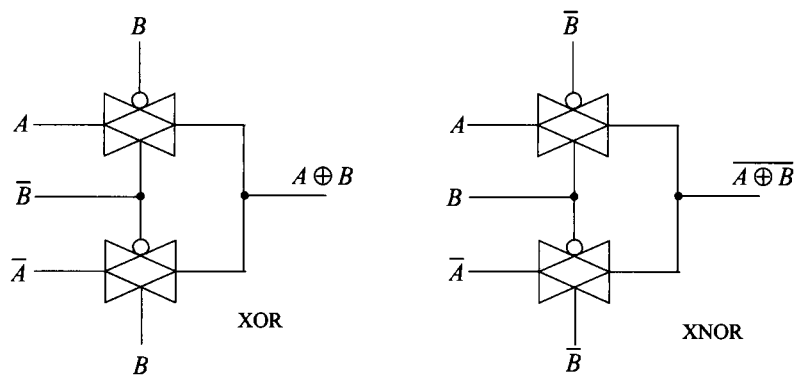


Figure 13.12 TG implementation of XOR/XNOR gate.

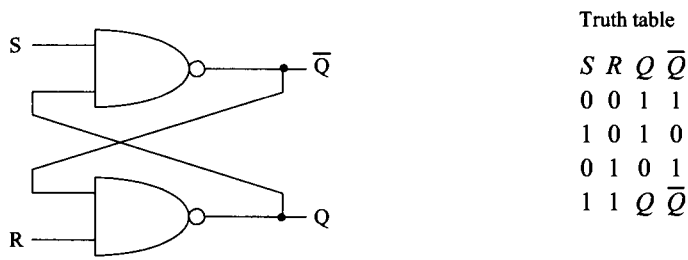


Figure 13.13 Set-reset latch made using NAND gates.

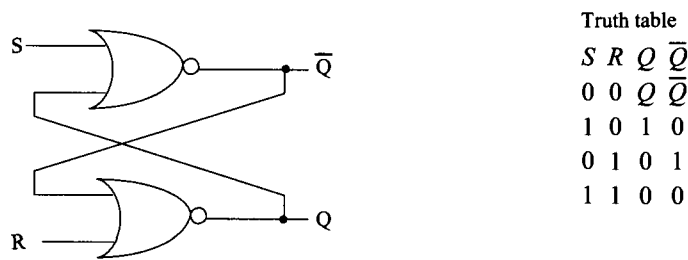


Figure 13.14 Set-reset latch made using NOR gates.

An Arbiter

One of the uses of the NAND latch is in an arbitration circuit (called an arbiter, Fig. 13.15). This circuit can be very useful in asynchronous circuit design or in clock synchronization circuits (see Ch. 19). To understand the operation of the arbiter in Fig. 13.15, let's consider the case when both In1 and In2 are low. Both NAND gates' outputs are high and the two inverters' outputs, that is Out1 and Out2, are low. When In1 goes high, the output of X1 goes low, while the output of X2 remains high. This causes Out1 to go high and Out2 to remain low. Notice that with the output of X1 going low the power supplied to the PMOS in the inverter connected to Out2 is removed (making it impossible for Out2 to go high). When In2 goes high, while In1 is already high, the fact that the output of X1 is a low keeps the output of X2 high and Out2 low. When In1 goes low, with In2 high, Out1 goes low and Out2 goes high. The usefulness of this circuit occurs when In1 and In2 transition high at nearly the same times. The way the inverters are powered by the NAND gates makes it impossible for both Out1 and Out2 to go high at the same time. *Thus an arbiter is useful for determining which input arrives first.*

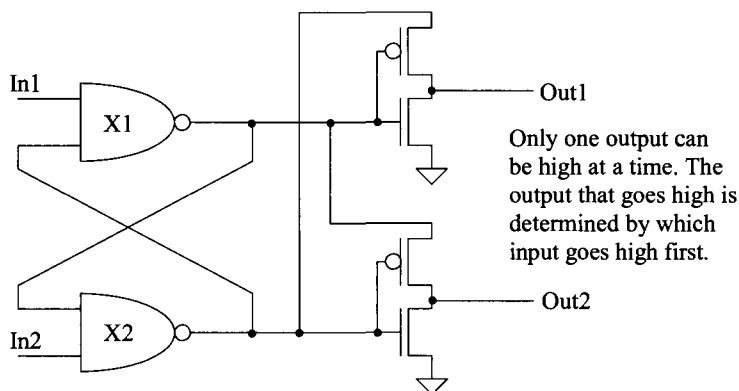


Figure 13.15 An arbiter made using NAND gates.

To ensure that the arbiter makes the decision quickly (there isn't a long delay or *metastability* when the two inputs are arriving at the same time), two inverters can be added in series with the NAND gate outputs. This addition of inverters simply increases the gain of the NAND gates (their VTCs get steeper at the switching point, see Fig. 12.3 in the last chapter).

Flip-Flops and Flow-through Latches

A flow-through latch is a storage circuit whose output changes with a clock signal level (so a flow-through latch is often called a level-sensitive latch). A flip-flop (FF) is a storage circuit that changes states on the rising or falling edge of a clock signal.

Most FFs in CMOS IC design are based on the cross-coupled inverters seen in Fig. 13.16. Also seen in this figure are the VTCs for the two inverters. It's characteristics are drawn with its input on the x-axis and its output on the y-axis (the normal curves as

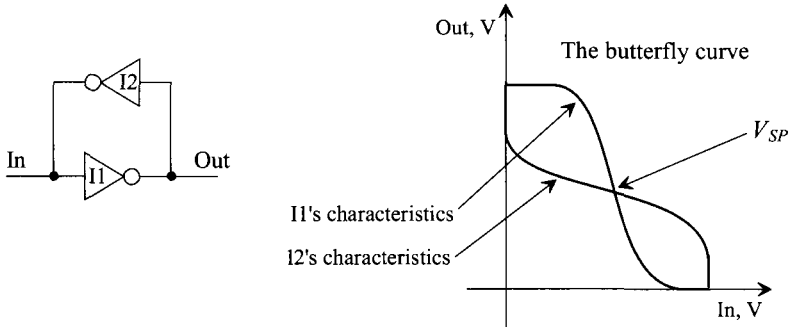


Figure 13.16 A cross-coupled inverter (a latch) and the input output characteristics.

seen in Fig. 11.2 back in Ch. 11). I2's input is labeled Out in the figure while its output is labeled In (and so its input is drawn as the y-axis and its output is drawn as the x-axis). The big concern when designing a FF with cross-coupled inverters is *metastability* (meaning changing stability). When the input signal moves to the switching-point of the inverters, both In and Out will be at V_{SP} (a stable state). This isn't where we want the circuit to operate because V_{SP} isn't a valid logic level voltage. Driving the input higher or lower enables the positive feedback present in the cross-coupled inverters to drive the In/Out nodes to valid logic levels (the other stable states). To avoid metastability, we can 1) use longer length inverters (which increases the gains of the inverters at the cost of longer delays), 2) use two inverters in series with the outputs of each inverter in Fig. 13.16 (again, to boost the gain) so that three inverters are used between In and Out, 3) ensure the In signal is driven with good logic levels (and a low impedance source to overdrive the output of I2), or 4) add a switch on the output of I2 to disconnect it when the In signal is changing to ensure that the input signal doesn't have to "fight" with the output of I2 to drive the signal to a valid logic level (the input signal range must still swing to valid logic levels). Figure 13.17 shows how metastability can result in a long delay before the outputs move to valid logic states. In this simulation the two capacitors are charged to 500 mV. It takes approximately 2.5 ns for the In/Out signals to change states.

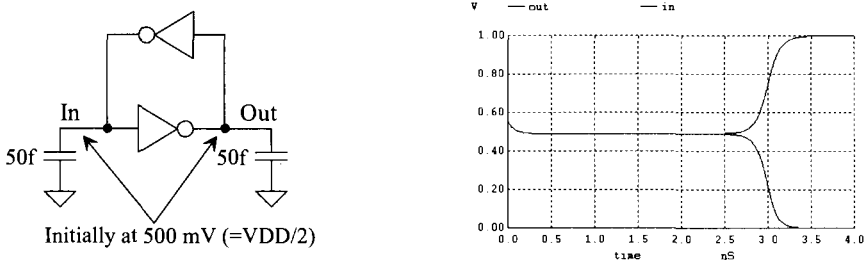


Figure 13.17 The delay associated with metastability.

Note that trying to move the switching point voltages of each inverter to differing levels makes the problem of metastability worse. When each inverter's V_{sp} is the same the gain around the loop is the highest (and so it's possible noise alone can cause the inverters to snap to a valid logic state).

Figure 13.18 shows a level-sensitive latch. When the clock signal is high, the data or D input passes through the NMOS PG and drives the cross-coupled inverters. The feedback inverter's lengths are made long to ensure that the source driving the D input can "overpower" the feedback inverter and allow the output to switch states. This circuit, while simple, has several fundamental issues. To begin, we know that an NMOS pass gate's output will swing from 0 to $V_{DD} - V_{THN}$. To optimize the noise margins, we lower the V_{sp} of I1. Here we reduce the width of the PMOS from 20 to 10. Another concern is the contention current that flows when the D input is fighting with the output of I2 for control of I1's input. By increasing the lengths in I2 to 10, we lower this wasted current.

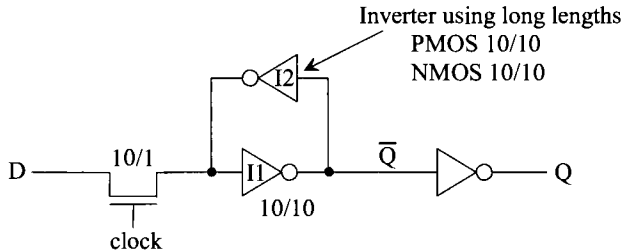


Figure 13.18 A level-sensitive latch.

Figure 13.19 shows the input and output simulation results for the latch in Fig. 13.18. When the clock signal is high, the input flows to the output (with a delay). When the clock signal goes low, the value on the D input is captured and remains on the Q output until the clock signal goes back high again. Notice that at 8 ns, when the clock signal goes high, the delay between the D input going high and the Q output going high is considerable.

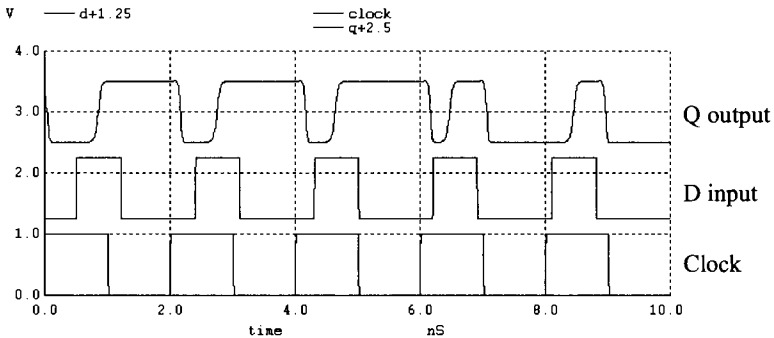


Figure 13.19 Simulating the level-sensitive latch in Fig. 13.18.

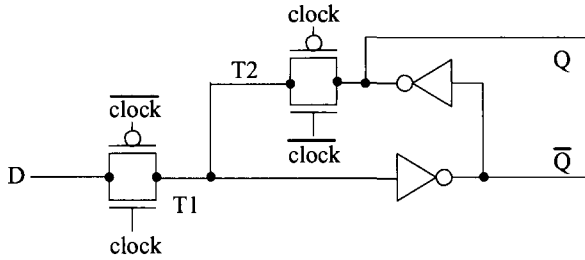


Figure 13.20 A higher performance level-sensitive latch.

Figure 13.20 shows a modification of the latch in Fig. 13.18. A TG is used on the input of the latch instead of a PG. This improves the noise performance of the circuit (at the cost of an additional clock signal, that is, the complement of clock). We've also added a TG in series with the feedback inverter. When T1 is on, the Q output follows the D input. When T1 is on, T2 is off so that the input doesn't have to fight with the feedback inverter. When T1 turns off, T2 turns on and the value of D at that instance is stored in the inverters. Figure 13.21 shows the simulation results using this latch with the input signals used to generate Fig. 13.19. Notice, for example, the improvement in performance over what's seen in Fig. 13.19 when the clock transitions high at 8 ns.

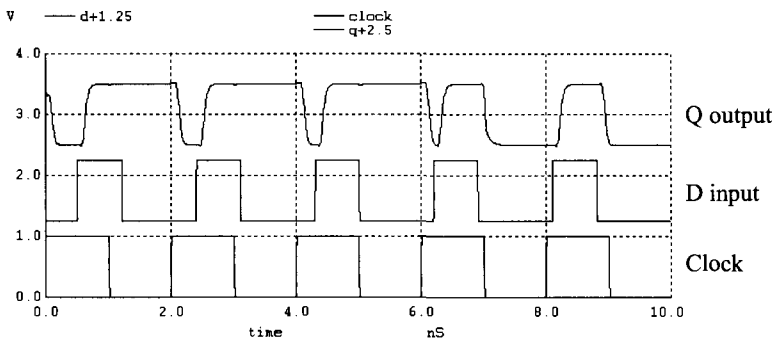


Figure 13.21 Simulating the level-sensitive latch in Fig. 13.20.

An Edge-Triggered D-FF

Notice that the value of the D input in the circuit seen in Fig. 13.20 is stored or captured when clock goes low. To implement an edge-triggered FF, we can use two of these level-sensitive latches in cascade. When the clock is low, the first stage tracks the D input and the second stage holds the previous output. When clock goes high, the first stage captures the input and transfers it to the second stage. The first stage is often called the “master” latch, while the second stage is the “slave” latch. Figure 13.22 shows an implementation of an edge-triggered D-FF. When clock is low, T1 and T4 are on. T2 and T3 are off. The D input flows through to point A and its complement to point B. When clock goes high, T1 and T4 shut off while T2 and T3 turn on. This causes the value of the D input, when

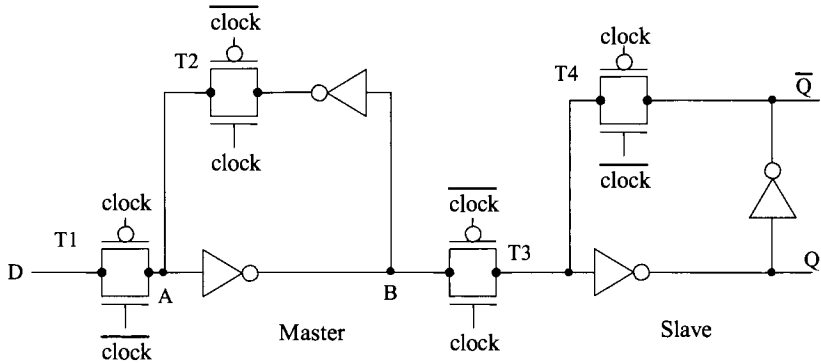


Figure 13.22 An edge-triggered D-FF.

clock transitioned high, to be captured and passed to the Q output. When clock goes back low, T1 and T4 turn on. The value of D on the Q output is then circulated around the two inverters. The Q output can change states again when clock goes back high. Figure 13.23 shows results of simulating this FF with SPICE. Note the output only changes on the rising edge of the clock signal.

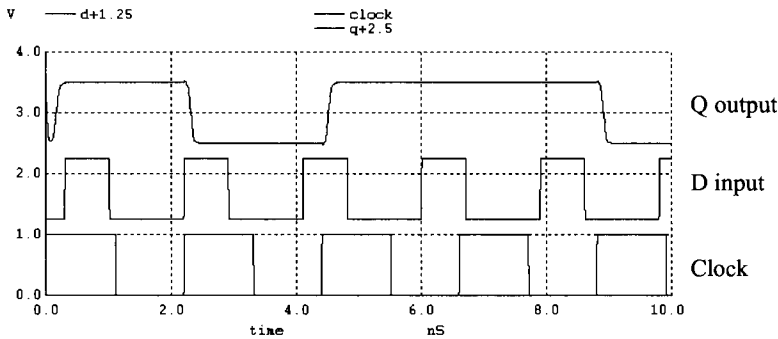


Figure 13.23 Simulating the operation of the edge-triggered FF in Fig. 13.22.

Figure 13.24 shows the addition of NAND gates to the circuit of Fig. 13.22 for clear and set inputs. When clear goes low, the outputs of N1 and N4 go high. If T3 is on or T4 is on, the outputs of the FF are forced to $Q = 0$ (and thus its complement is set to a 1). If the set input goes low, then Q is forced to a 1. If both clear and set are pulled low, then both the Q output and its complement are driven high. This feature can useful in some situations.

Note that it is **very important** for the clock signals we use to have quick rise times. If they don't, then the FF may not function correctly. Often the clock signals are applied to a string of inverters both to sharpen their transition times and to generate the complement clock required for the TGs.

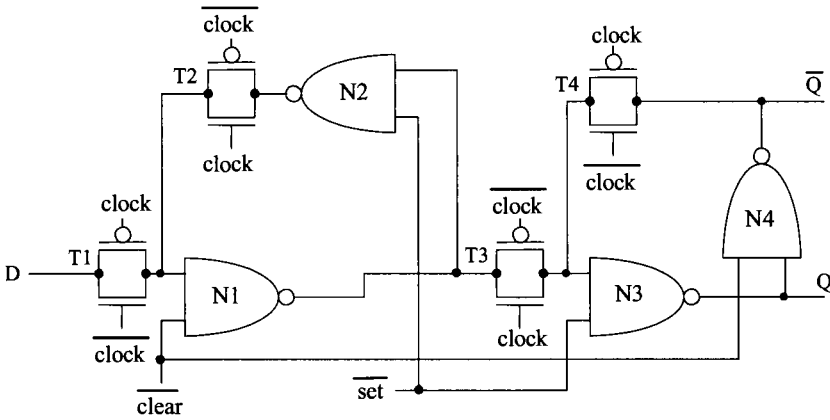


Figure 13.24 An edge-triggered FF with asynchronous set and clear.

Flip-Flop Timing

The data must be *set up* or present on the D input of the FF (see Fig. 13.22) a certain time before we apply the clock signal. This time is defined as the setup time of the FF. To understand the origin of this time, consider the time it takes the signal at D to propagate through T1 and the inverter to node B. Before the clock pulse can be applied, the logic level D must be settled on node B. Consider the waveforms of Fig. 13.25. The time between D going high (or low) and the clock rising edge is termed the setup time of the FF and is labeled, t_s .

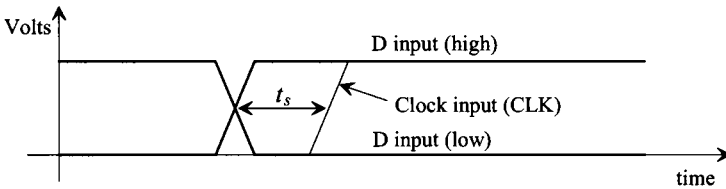


Figure 13.25 Illustrating D FF setup time.

The wanted D input must be applied t_s before the clock pulse is applied. Now the question becomes “How long does the wanted D input have to remain applied to, or *held* on, the input of the FF after the clock pulse is applied?” This time is called the hold time, t_h , and is illustrated in Fig. 13.26. Shown in this figure, t_h is a positive number. However, inspection of Fig. 13.22 shows that if the D input is removed slightly before the clock pulse is applied, node B will remain unchanged because of the propagation delay from the D input to node B. Analysis of this FF would yield a negative hold time. In other words, for the point B to charge to D, a time labeled t_s is needed. Once point B is charged, the D input can be changed as long as the clock signal occurs within t_h .

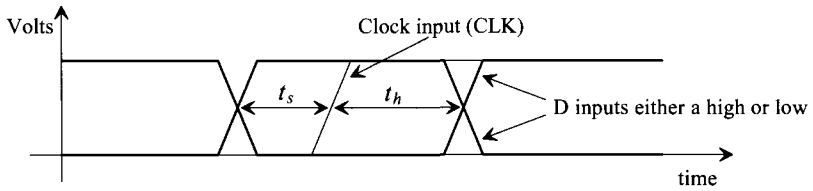


Figure 13.26 Illustrating D FF hold time.

One final important comment regarding the clock input of an FF is (again) in order. If the clock input risetime is slow, the FF will not function properly. There will not be an abrupt transition between the sets of transmission gates turning on and off. The result will be logic levels at indeterminate states. What is usually done to eliminate this problem is to buffer the clock input through several inverters. This has the effect of speeding up the leading and trailing edges of a slow input pulse and presenting a lower input capacitance on the clock input to whatever is driving the FF. The main disadvantage is the increase in delay times, t_{PHL} and t_{PLH} (defined by clock to output), of the FF. In general, the FF of Fig 13.22 should not be laid out without buffering the clock inputs.

The minimum pulse width of the clock, set, or clear inputs in Figs. 13.22 and 13.24 is labeled t_w . The minimum width is determined by the delay through (referring to Fig. 13.20) two NAND gates and a TG. The last timing definition we will comment on here is the recovery time, that is, the time between removing the set or clear inputs and a valid clock input. This variable is labeled t_{rec} .

13.4 Examples

In this section we give examples of delay calculations and comments on how to decrease the delays and the associated performance costs. Throughout this section we use the 50 nm short-channel process.

Example 13.3

Estimate the input capacitance and the delay through the circuit seen in Fig. 13.27. Compare the hand calculation estimates to simulation results.

Using the information seen in Table 10.2, we can calculate the first inverter's input capacitance, and thus the input capacitance of the circuit,

$$C_{in1} = \frac{3}{2} \cdot (C_{oxn1} + C_{oxp1}) = 0.938 \text{ fF} + 1.875 \text{ fF} = 2.81 \text{ fF}$$

Since the widths of the second inverter are 10 times larger than the widths of the first inverter, the input capacitance of the second inverter is 28.1 fF.

The delay through the first inverter, that is, the delay from the node labeled "In" to the node labeled "N1" in Fig. 13.27 is calculated as

$$t_{PHL1} + t_{PLH1} = 0.7 \cdot (3.4k + 3.4k) \cdot (0.625 + 1.25 + 9.38 + 18.75) \text{ fF} = 143 \text{ ps}$$

noting, because $t_{PHL} = t_{PLH}$, that $t_{PHL1} = t_{PLH1} = 71.5 \text{ ps}$.